

GridScript

Design document

A small interpreter for scripting a 2D game actor

Contributors

Nabil Ismail Abdulkadir

UG22CSC1047

Interpreter and semantics

Ahmad Auwal Abubakar

UG22CSC1075

Parser and grammar

Abubakar Muhammad Sulaiman

UG22CSC1046

Lexer and token rules

Rukayya Musbahu Imam

UG22CSC1040

AST and lexical scoping

Muhammad Salisu

UG20CSC1005

Tests and report

Course	CSC 4207 (Organization of Programming Languages)
Group name / number	Group 1
Members & reg. numbers	Nabil Ismail Abdulkadir (UG22CSC1047); Ahmad Auwal Abubakar (UG22CSC1075); Abubakar Muhammad Sulaiman (UG22CSC1046); Rukayya Musbahu Imam (UG22CSC1040); Muhammad Salisu (UG20CSC1005)
Submission	(one email, zipped, to salisu.abdul@kustwudil.edu.ng)

1. Overview

GridScript is a small imperative language for scripting the behaviour of a game actor (a player or NPC) moving on a 2D grid. The language deliberately contains only a handful of constructs:

- arithmetic expressions
- variables and assignment
- if / while control flow
- user-defined functions with parameters and return
- built-in actor actions (`step_forward()`, `turn_left()`, ...)
- a print statement for output

Every construct maps onto a topic from the course:

Construct	Course topic
lexer (tokens)	regular expressions & finite automata
parser / grammar	context-free grammars
interpreter rules	operational semantics
functions, params, shadowing	names, binding & scoping
dynamic typing rules	type systems (runtime types)

2. Lexical structure (regular expressions)

The lexer reads the source character by character and emits tokens. Whitespace (space, tab, newline) separates tokens and is otherwise ignored. Comments start with `//` and run to the end of the line.

Token	Lexeme / regular expression	Notes
KEYWORD	set if then else end while do def return print true false	reserved; may not be used as identifiers
IDENT	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	variable / function names
NUMBER	<code>[0-9]+</code>	non-negative integers
STRING	<code>"[^"]*"</code>	double-quoted text
OP	<code>= == != < > + - * /</code>	single- and two-character operators
LPAREN	<code>(</code>	
RPAREN	<code>)</code>	
COMMA	<code>,</code>	
COMMENT	<code>//[^\n]*</code>	stripped by the lexer, produces no token
EOF	end of file	

Notes:

- Maximal munch: the longest matching token is taken. $\geq$ -style operators are deliberately absent, so $>$ then $=$ is a syntax error rather than an accident.
- Keyword vs identifier: lexed as IDENT first, then looked up in the keyword table; this keeps the DFA small (one letter pattern) and the keywords table-driven.
- Numbers: the pattern $[0-9]^+$ accepts multi-digit integers such as 100 and 25.

3. BNF grammar

```

<Program>      ::= <StatementList> EOF

<StatementList> ::= <Statement>
                    | <Statement> <StatementList>

<Statement>    ::= "set" IDENTIFIER "=" <Expression>
                    | "print" <Expression>
                    | "if" <Expression> "then" <StatementList>
                      ["else" <StatementList>] "end"
                    | "while" <Expression> "do" <StatementList> "end"
                    | "def" IDENTIFIER "(" <ParamList> ")"
                      <StatementList> "end"
                    | "return" <Expression>
                    | <Call>

<Call>         ::= IDENTIFIER "(" <ArgList> ")"

<ParamList>    ::= IDENTIFIER ("," IDENTIFIER)* | ε
<ArgList>      ::= <Expression> ("," <Expression>)* | ε

<Expression>   ::= <Comparison>
<Comparison>   ::= <Arithmetic> [ ( "=" | "!=" | "<" | ">" ) <Arithmetic> ]
<Arithmetic>   ::= <Term> ( ( "+" | "-" ) <Term> ) *
<Term>         ::= <Factor> ( ( "*" | "/" ) <Factor> ) *
<Factor>       ::= NUMBER | STRING | "true" | "false" | IDENTIFIER
                    | <Call> | "(" <Expression> ")" | "-" <Factor>

```

Grammar design notes:

- Precedence is encoded in the grammar: $*$ / bind tighter than $+$ -, which bind tighter than comparisons. $2 + 3 * 4$ parses as $2 + (3 * 4)$.
- else is optional; an else always attaches to the nearest unmatched if.
- Function calls are expressions (<Call> appears inside <Factor>), so calls can be nested or combined with arithmetic, e.g. `set health = health - scale(2, 3)`. A call used on its own as a <Statement> (e.g. `step_forward()`) simply discards the value.
- A single (optional) comparison, not a chain, avoids nonsense such as $1 < 2 < 3$.
- ArgList / ParamList are comma-separated lists and may be empty, so both `scale(2, 3)` and `turn_left()` parse.
- The grammar is LL(1)-friendly and is parsed by recursive descent: one function per non-terminal, with one-token lookahead to pick between IDENT (variable reference), IDENT "(" (call), and literal tokens.

4. Values and typing rules (dynamic typing)

Values carry their type at run time. There are no type annotations in the source.

Type	Examples
Number	0 25 100
String	"grid" "low health"
Boolean	true false

Dynamic typing rules for operators (checked at run time, else a `TypeError`):

Operator	Allowed operands	Result
+	Number + Number	Number
+	String + String	String (concatenation)
- * /	Number, Number	Number
< >	Number, Number	Boolean
== !=	any two values of the same type	Boolean (by value)
==	values of different types	Boolean false (not an error)
!=	values of different types	Boolean true (not an error)

Division by zero is a run-time error. `if/while` conditions must evaluate to a Boolean; anything else is a run-time `TypeError` (`if 5 then ...` is rejected). We do not coerce numbers to booleans. That keeps the typing rules simple to state and to test, and it gives us a ready-made invalid test case. Booleans are not Numbers: `true + 1` fails with `cannot add Boolean and Number`, and `1 == true` is `false` because the operands have different types.

5. Scoping rule (static / lexical) and rationale

Only function calls create new scopes. `if/while` bodies execute in the environment of the surrounding code (no block scoping). Three rules:

1. `set x = e` always binds/updates `x` in the current environment; it never reaches into an outer scope.
2. Reading a name resolves it lexically: current environment first, then its enclosing environments, out to the global environment.
3. A function body executes in a fresh environment whose `**parent` is the environment where the function was `*defined*` (not where it was called).

Consequences that are easy to state, implement, and test:

- A parameter or local inside a function shadows a global of the same name; assigning inside the function never clobbers the global.
- Recursive calls each get their own frame.
- Global mutable state is shared only by top-level statements; a function can read a global but must be given values through parameters and give results back through `return`.

Why static (lexical) scoping? Because a function's meaning can be understood from its text alone: you read `def f(x) ... end` and know which `x` it means without tracing call history. Dynamic scoping would make the same function behave differently depending on who called it, which for a game-script language would make actors behave unpredictably. It also matches the course material on names and

binding, where we can explain environment chains and shadowing concretely.

Why dynamic typing? GridScript is a small scripting language; scripts talk about health as a number one moment and an item name as a string the next. Dynamic typing keeps the grammar and parser small (no type annotations to parse) and pushes the type checks into the interpreter, where each construct has one simple run-time rule. The trade-off (errors surface at run time rather than before) is acceptable for short scripts, and it is why the test suite includes type-error cases.

6. Operational semantics (how constructs evaluate)

We write σ for the current environment (a name $\rightarrow$ value mapping, resolved lexically) and v for a run-time value. $\langle e, \sigma \rangle \Downarrow v$ reads "expression e evaluates to v in σ ". $\sigma[x \mapsto v]$ means " σ updated/declared locally with $x \mapsto v$ ". One rule per construct:

Expression evaluation

$\langle \text{NUMBER } n, \sigma \rangle \Downarrow n$	$\langle \text{STRING } s, \sigma \rangle \Downarrow s$
$\langle \text{true}, \sigma \rangle \Downarrow \text{true}$	$\langle \text{false}, \sigma \rangle \Downarrow \text{false}$
$\langle x, \sigma \rangle \Downarrow \sigma(x)$ (lexical lookup; error if unbound)	
$\frac{\langle e_1, \sigma \rangle \Downarrow v_1 \quad \langle e_2, \sigma \rangle \Downarrow v_2 \quad v_1, v_2 \text{ are Numbers}}{\langle e_1 + e_2, \sigma \rangle \Downarrow v_1 + v_2}$ (same pattern for $-$, $*$, $/$, $<$, $>$)	
$\frac{\langle e_1, \sigma \rangle \Downarrow v_1 \quad \langle e_2, \sigma \rangle \Downarrow v_2 \quad v_1, v_2 \text{ are Strings}}{\langle e_1 + e_2, \sigma \rangle \Downarrow v_1 \text{ concatenated with } v_2}$	

Comparisons evaluate both sides to values of the permitted types and yield a Boolean (section 4). A parenthesised expression evaluates its contents; unary $-$ negates its operand (Number only).

Assignment

$\langle e, \sigma \rangle \Downarrow v$	
$\langle \text{set } x = e, \sigma \rangle \Downarrow \sigma[x \mapsto v]$	(declare or update x in the current environment only; see section 5)

Output

$\langle e, \sigma \rangle \Downarrow v$	
$\langle \text{print } e, \sigma \rangle \Downarrow \sigma$	(side effect: write v to the console)

Conditional (else optional; absent else behaves as an empty body)

$\langle c, \sigma \rangle \Downarrow \text{true}$	$\langle S_1, \sigma \rangle \Downarrow \sigma'$
$\langle \text{if } c \text{ then } S_1 \text{ else } S_2 \text{ end}, \sigma \rangle \Downarrow \sigma'$	
$\langle c, \sigma \rangle \Downarrow \text{false}$	$\langle S_2, \sigma \rangle \Downarrow \sigma'$
$\langle \text{if } c \text{ then } S_1 \text{ else } S_2 \text{ end}, \sigma \rangle \Downarrow \sigma'$	

Loop

```

{c, σ} ↦ false
-----
{while c do S end, σ} ↦ σ

{c, σ} ↦ true   {S, σ} ↦ σ'   {while c do S end, σ'} ↦ σ'
-----
{while c do S end, σ} ↦ σ'

```

User-defined function call. `def f(p1, ..., pn) S end` binds the name `f` to a closure $(p_1, \dots, p_n, S, \sigma_{\text{def}})$: the parameter list, the body, and the environment at definition time. A call creates a fresh environment whose parent is σ_{def} , binds each parameter to the evaluated argument, then runs the body:

```

{f, σ} ↦ closure (p1, ..., pn, S, σdef)
{a1, σ} ↦ v1 ... {an, σ} ↦ vn
σnew = child of σdef with pi ↦ vi
{S, σnew} ↦ σ'   and body executes "return e" ⇒ {e, σ'} ↦ v
-----
{f(a1, ..., an), σ} ↦ v

```

`return e` halts the function immediately and yields the value of `e`. If control reaches the end of a body without a `return`, the call evaluates to the Number 0 (this is harmless for side-effect calls such as `step_forward()`).

Built-in actor actions. The interpreter owns a small "world" object (actor position, facing direction, health). Built-ins mutate it and return a Number:

Built-in	Effect	Returns
<code>step_forward()</code>	move one cell in the current facing direction	1
<code>turn_left()</code> / <code>turn_right()</code>	change facing	1
<code>use_potion()</code>	restore health	1 if a potion is held, else 0

The grid world is implemented in `src/builtins.py` and keeps the actor's position, facing direction, health, and potion count. The language core (lexer, parser, and the interpreter rules above) covers the course concepts; the built-ins give scripts something visible to act on.

7. Sample programs

7.1 Valid: patrol loop (assignment, arithmetic, loop, conditional, calls)

```

// The actor patrols, taking damage each step, until health runs out.
set health = 100
set damage = 25

while health > 0 do
  step_forward()
  set health = health - damage
  if health < 50 then
    print "low health"
  end
end
print "patrol complete"

```

Output: low health twice (once after health falls to 25, once after it reaches 0), then patrol complete. At the end health is 0 (it never goes negative because the loop exits as soon as `health > 0` is false).

7.2 Valid: user-defined function, parameters and return

```
def scale(value, factor)
  return value * factor
end

set bonus = 4
print scale(3, bonus)      // prints 12
print scale(bonus, bonus) // prints 16
```

7.3 Valid: static scoping / shadowing demo (use this in the report!)

```
set x = 100

def hide()
  set x = 7      // local: binds x in hide's own frame
  return x
end

print x          // prints 100
print hide()     // prints 7
print x          // prints 100 ← global x is untouched
```

This program shows that the interpreter implements static scoping: the `set x = 7` inside `hide` cannot leak to the global `x`.

7.4 Valid: precedence and types

```
print 2 + 3 * 4      // prints 14  (* binds tighter than +)
print (2 + 3) * 4    // prints 20
set name = "grid"
print name + " script" // prints grid script (string concatenation)
print 1 == 1         // prints true
print 1 == 2         // prints false
print "a" == 1       // prints false (different types never equal)
print 1 == true      // prints false (Number and Boolean differ)
print 1 != true      // prints true
```

7.5 Invalid: syntax errors (lexer/parser must reject)

```
set health = 100$
```

Expected: lexer error, unexpected character '\$'.

```
if health < 50      // missing "then"
  use_potion()
end
```

Expected: parser error, expected 'then'.

```
set x = (2 + 3
```

Expected: parser error, expected RPAREN at end of input.

7.6 Invalid: run-time errors (interpreter must reject with a clear message)

```
print mana          // undefined variable
```

Expected: run-time error, undefined variable 'mana'.

```
print 1 / 0
```

Expected: run-time error, division by zero.

```
if 5 then
  print "nope"
end
```

Expected: run-time `TypeError`, if condition must be a `Boolean`, got `Number`.

```
set hp = "high"
print hp + 100
```

Expected: run-time `TypeError`, cannot add `String` and `Number`.

```
print true + 1
```

Expected: run-time `TypeError`, cannot add `Boolean` and `Number`.

8. Testing approach

The test suite runs programs and checks two things: the output for valid programs (7.1 to 7.4) and the error kind and message for invalid programs (7.5 to 7.6). Tests are organised as:

- `tests/valid/*.gsc`: each pairs a script with an expected-output file.
- `tests/invalid/*.gsc`: each pairs a script with an expected error line.

Coverage checklist (ties back to the marking rubric):

- [x] lexer: keywords, identifiers, numbers, strings, operators, comments, errors
- [x] parser: precedence, optional `else`, nested `if`, empty argument lists
- [x] interpreter: assignment, arithmetic, strings, `if`, `while`, functions,

`return`, recursion

- [x] scoping: shadowing, globals preserved (7.3), recursion frames
- [x] typing: valid mixed-type `==`, and each `TypeError` path
- [x] two or more invalid programs of each kind (syntax and run-time)

9. AI-use disclosure

The brief permits AI tools for minor help, such as debugging an error message, but not for generating the grammar, semantic rules, or core code. The group used NotebookLM to brainstorm language themes and prepare slides for an initial group meeting, plus AI help to clarify one Python error message during development.

The group wrote and reviewed the grammar, typing and scoping rules, operational semantics, and core implementation itself.

10. Group contributions

Member	Contribution	Course topic
Nabil Ismail Abdulkadir	Interpreter, built-ins, and errors	Operational semantics and type systems
Ahmad Auwal Abubakar	Parser and BNF grammar	Context-free grammars
Abubakar Muhammad Sulaiman	Lexer and token rules	Regular expressions and finite automata
Rukayya Musbahu Imam	AST, environments, and scoping rules	Names, binding, and scoping
Muhammad Salisu	Test suites, test cases, and report	Testing across all topics

GridScript, CSC 4207 group project.